Подготовка текста и проверка результата

Предобработка, косинусное сходство, метрики качества

О чём эта часть

1

Предобработка

Как очистить текст перед векторизацией

2

Сходство

Как понять, что два вектора похожи

3

Качество

Как проверить, что модель работает

4

Типы векторов

Разреженные и плотные: в чём разница

Конвейер предобработки

Исходный текст	**«Коты СПЯТ на диване!»**
1. Нижний регистр	«коты спят на диване!»
2. Убрать знаки	«коты спят на диване»
3. Разбить на токены	коты · спят · на · диване
4. Убрать стоп-слова	коты · спят · диване
5. Лемматизация	**кот · спать · диван**

Цель: чтобы «Коты» и «кот» стали одним словом, а лишние знаки не попали в словарь.

Стоп-слова

Частые слова-«клей»

и	в	на	что
это	с	по	как

Они есть почти в каждом документе и ничего не говорят о теме. Если их убрать, словарь и векторы становятся короче.

Но будьте осторожны

В некоторых списках стоп-слов есть «не».

«хороший фильм» и «не хороший фильм» — противоположные отзывы. Если убрать «не», модель их перепутает.

Поэтому список стоп-слов подбирают под задачу.

Лемматизация и стемминг

Лемматизация

Приводит слово к словарной форме: «коты», «кота», «котов» → «кот»; «спали» → «спать».

Нужен словарь языка.

Стемминг

Отрезает окончание по правилам и оставляет «основу».

Быстро, но получается обрубок и может потеряться смысл.

кот **кота** **коту** **коты** **котов** → **кот**

Пять форм — одно место в словаре.

Коллизии при стемминге

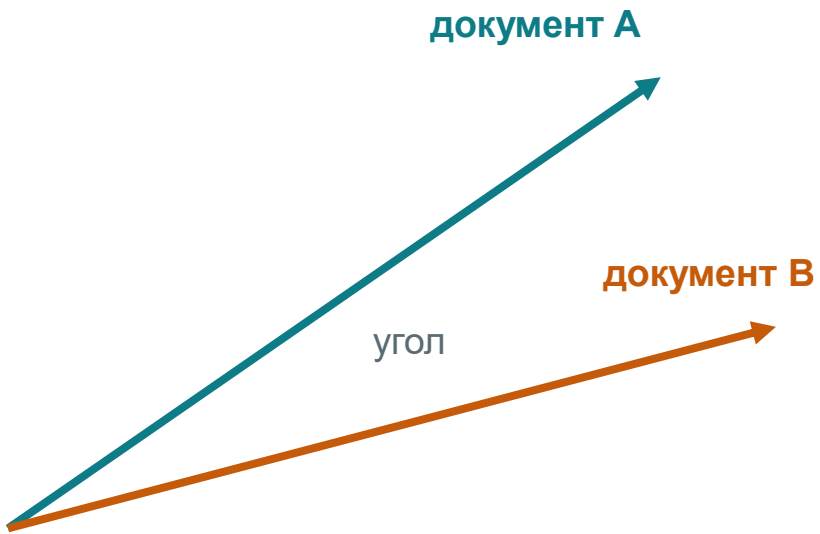
Пара слов	Стемы	Что произошло
лес / лесной	лес / лесн	✗ разные (правильно — прилагательное должно отличаться)
пол / полка	пол / полк	✗ разные (но они и не связаны)
лес / лесник	лес / лесник	✗ разные — хотя «лесник» родственно «лес»
ухо / услышать	ух / услыш	✗ разные
бег / бегство	бег / бегств	✗ разные — хотя родственные

Потери при стемминге

Тип потери	Пример	Что происходит
Супплетивизм	хороший ↔ лучше	Формы одного слова → разные стемы
Синонимия	хороший ↔ отличный	Синонимы → разные стемы
Супплетивизм глаголов	иду ↔ шёл	Один глагол → разные стемы
Ложная коллизия	лес ↔ леска (редко)	Разные слова → один стем
Полисемия	замок (дверь) ↔ замок (здание)	Разные смыслы → один стем

Задача	Стемминг ОК?
Спам-фильтр	✅ Достаточно — смысл не критичен, важны отдельные слова
Тематическая классификация (спорт/политика)	✅ Достаточно
Анализ тональности	❌ Плохо — «хороший» и «лучше» станут разными признаками
Поиск по сайту	⚠️ Средне — найдёт «лесной» по запросу «лес», но пропустит «лесник»
Машинный перевод	❌ Плохо — теряется грамматическая информация
NER (имена, организации)	❌ Плохо — нужно знать части речи
Семантический поиск	❌ Плохо — синонимы не сближаются, нужен BERT

Как измерить, что векторы похожи



1 в одну сторону: очень похожи

0 под прямым углом: ничего общего

–1 в разные стороны: противоположны

Чем меньше угол, тем больше косинус.

Косинус на примере BoW

$$\cos(a, b) = (a \cdot b) / (|a| \cdot |b|)$$

Скалярное произведение, делённое на произведение длин векторов.

Пара документов	Общие слова	Косинус
«кот ест рыбу» и «пёс ест кость»	ест	**0.33**
«кот ест рыбу» и «кот кот спит»	кот	**0.52**
«пёс ест кость» и «кот кот спит»	нет	**0**

У двух разных one-hot векторов косинус всегда 0: поэтому «кот» и «собака» никак не связаны

Косинус на Python

```
import numpy as np

def cos(a, b):
    return a @ b / (np.linalg.norm(a)
                    * np.linalg.norm(b))

cos(d1, d2)   # 0.33
```

Длинный и короткий

Если абзац повторить дважды, все счётчики удвоятся. Тема не изменилась, и косинус по-прежнему 1.

Для текстов косинус удобнее обычного расстояния: он сравнивает направление, а не длину.

Проверяем на новых данных



Обучение

Модель учится на этих текстах: строит словарь, считает вероятности.

Проверка

Тексты, которых модель не видела. Только по ним честно оценивают качество.

Проверять на обучающих данных — всё равно что сдавать экзамен по уже известным билетам.

Матрица ошибок

	Предсказано: спам	Предсказано: не спам
Реально спам	**16** спам найден	**4** спам пропущен
Реально не спам	**2** ложная тревога	**78** всё верно

100 писем

20 из них спам, 80 обычные.

Две ошибки разного рода: пропущенный спам и потерянное нужное письмо.

Accuracy, precision, recall

0.94

Accuracy

доля верных ответов: (16 + 78) / 100

0.89

Precision

из найденного как спам — настоящий спам: 16 / (16 + 2)

0.80

Recall

из всего спама найдено: 16 / (16 + 4)

0.84

F1

среднее precision и recall в одном числе

Одной accuracy мало: если спама 1%, модель «всегда не спам» получит 99% и не найдёт ни одного спама (recall = 0).

Всё вместе на Python

```
X_train, X_test, y_train, y_test = \
    train_test_split(texts, labels, test_size=0.2)

model = make_pipeline(TfidfVectorizer(),
                      MultinomialNB())
model.fit(X_train, y_train)

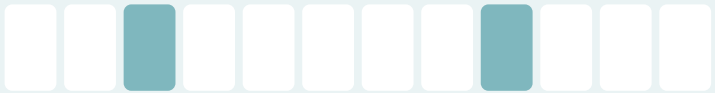
print(classification_report(
    y_test, model.predict(X_test)))
```

1. Разбить данные
2. Обучить на 80%
3. Проверить на 20%

Разреженные и плотные векторы

Разреженный

one-hot, BoW, TF-IDF



Длина равна размеру словаря (например, 50 000). Почти все числа — нули. Каждое число — конкретное слово, это легко объяснить.

Плотный

эмбеддинги (Word2Vec, BERT)



Длина 100–768 чисел, нулей почти нет. Смысл распределён по всем числам, зато близкие слова получают близкие векторы.

Что запомнить

1. Текст чистят перед векторизацией: регистр, знаки, стоп-слова, лемматизация.
2. Косинус показывает, насколько похожи два вектора: от 0 до 1 для счётчиков слов.
3. Качество проверяют на тексте, которого модель не видела: precision, recall, F1.
4. Разреженные векторы понятны, плотные несут смысл.

Дальше: логистическая регрессия, семантический поиск, RAG